Big Data Introduction

What is *Data* in Data Engineering?

Simple Definition

Data = Raw facts or information that can be stored, processed, and used

Examples:

- Numbers → `100, 200`
- Text → `"Ram", "Order placed"`
- Dates → `2026-04-03`
- Logs → `"User logged in"`

But as a Data Engineer… think deeper

Data = Anything that can be captured from a system and moved, stored, and transformed

---

Real Understanding (VERY IMPORTANT)

Data in data engineering is not just "values" — it is:

1 Generated

- User actions (clicks, purchases)
- Systems (logs, sensors)

2 Stored

- Files (CSV, JSON)
- Databases

3 Processed

- Cleaned
- Transformed

4 Used

- Analytics

- Machine learning
- Business decisions

📊 Types of Data (Industry View)

🧱 Structured Data

- Tables (rows & columns)
- Example: Database table

📄 Semi-Structured Data

- JSON, XML
- Flexible format

🌫 Unstructured Data

- Images, videos, audio, logs

⚙ Internal Perspective (Think Like Engineer)

🧠 Data always exists in **3 forms**:

Form	Example
In Memory	Python objects (`list`, `dict`)
In Storage	Files / Database
In Transit	Streams / APIs

🔥 This is **VERY IMPORTANT**

🔄 Example (Real Data Flow)

User places order:

```
User clicks "Buy"
   ↓
Data generated → {"user": "Ram", "item": "Phone"}
   ↓
Stored → database / file
   ↓
```

```
Processed → analytics system
   ↓
Used → dashboard / ML model
```

Real-World Use Cases

- Amazon → user behavior data
- Banking → transactions
- YouTube → watch history
- Swiggy → order + delivery data

Interview Insight

If interviewer asks:

 “What is data?”

Don’t say just “raw facts”

Say:

 “Data is raw or processed information generated by systems or users, which can be stored, processed, and used to derive insights. In data engineering, we focus on moving and transforming this data across systems efficiently.”

⚠ Common Mistake

 Thinking data = only numbers or tables

✔ Data = **anything usable (logs, clicks, images, etc.)**

Final Master Line

 Data = Raw input → becomes useful output after processing

🧠 What is Big Data?

🔹 Simple Definition

🔹 **Big Data = Data that is too large, fast, or complex for traditional systems to handle**

🧠 Real Understanding (Think Like Engineer)

🔹 It's not just "large data"

🔹 Big Data = when **normal tools (like simple Python, Excel, basic DB)** 🔹 cannot process efficiently

---

⚡ The 3 V's of Big Data (VERY IMPORTANT)

1️⃣ Volume (Size)

- Huge amount of data
- TB, PB, even EB

🔹 Example:

- Billions of user records

2️⃣ Velocity (Speed)

- Data generated very fast

🔹 Example:

- Live transactions
- Real-time logs

3️⃣ Variety (Different Types)

- Structured + Semi + Unstructured

🔹 Example:

- Tables + JSON + images + videos

- Users: millions
- Data generated:
  - Posts
  - Likes
  - Comments
  - Videos

 Per day = **massive + fast + different formats**

➡ This is **Big Data**

⚙ Why Traditional Systems Fail

Tool	Problem
Excel	Crashes / limited rows
Normal DB	Slow queries
Single machine	Memory limitation

Big Data Solution

 We use **distributed systems**

- Data split across multiple machines
- Processed in parallel

- Apache Hadoop
- Apache Spark
- Kafka

□ Internal Working Idea (Important)

□ Instead of:

```
1 machine → process all data □
```

□ We do:

```
100 machines → each processes small part □
```

➡ Combine results → final output

□ Practical Use Cases

- Netflix → recommendations
- Amazon → product suggestions
- Banking → fraud detection
- Uber → real-time location tracking

□ Interview Insight (Strong Answer)

If asked:

□ "What is Big Data?"

Say:

□ "Big Data refers to datasets that are too large, fast, and complex for traditional systems to process efficiently. It is characterized by volume, velocity, and variety, and requires distributed computing frameworks like Hadoop or Spark for processing."

⚠ Common Mistakes

❌ Big data = only large size ✅ It's about **size + speed + complexity**

□ Final Master Line

□ **Big Data = Beyond capacity of traditional systems**

□ Why Traditional Systems Fail in Big Data?

□ One-Line Answer

□ **Because they are not designed to handle massive scale, speed, and complexity**

□ Core Problem

Traditional systems = □ **Single machine + limited resources**

Big Data = □ **Huge + fast + complex**

➡ **Mismatch → failure**

⚙ Detailed Reasons (VERY IMPORTANT)

1 Limited Storage (Volume Problem)

- Traditional DB / systems run on **one machine**
- Storage is limited (GBs or few TBs)

□ But Big Data:

- TBs → PBs → EXABYTES

□ Result: Cannot store all data

---

2 Limited Processing Power

- One CPU processes everything
- Slower when data grows

 Example:

```
1 million rows → OK
1 billion rows → very slow 
```

---

3 No Parallel Processing

- Traditional systems process **sequentially**

 Big Data needs:

- Parallel processing (multiple machines working together)

 Without it → huge delay

---

4 Memory Constraints

- RAM is limited

 Big Data:

- Cannot fit into memory

 Result:

- Crashes
- Disk swapping → very slow

---

5 Poor Handling of Variety

Traditional systems:

- Designed for **structured data (tables)**

 Big Data:

- JSON, logs, images, videos

 Result:

- Cannot process different formats efficiently

---

6 Scalability Problem (VERY IMPORTANT)

Traditional scaling:

 Vertical scaling (increase RAM/CPU)

 Problems:

- Expensive
- Has a limit

Big Data Solution

 Horizontal scaling:

```
1 machine 
↓
100 machines working together 
```

Internal View (Think Like Engineer)

Traditional System:

```
Data → Single Server → Process → Output
```

Big Data System:

```
Data → Split → Multiple Nodes → Parallel Processing → Combine → Output
```

Real Example

Banking System

- Millions of transactions per second

Traditional system:

 Slow

 Risk of failure

Big Data system: Distributed processing Real-time analysis

🧠 Interview Insight (Important Answer)

🎤 “Why do traditional systems fail?”

🗣 “Traditional systems fail in Big Data scenarios because they rely on single-machine processing with limited storage, memory, and CPU. They cannot scale horizontally or handle high volume, velocity, and variety efficiently, leading to performance bottlenecks and system failures.”

⚠ Common Mistake

❌ Saying “because data is large”

✔ Say **specific reasons**:

- No parallelism
- Limited resources
- Poor scalability

🔥 Final Master Line

🔥 **Traditional systems fail because they scale vertically, while Big Data needs horizontal scaling**

🧠 What are the V’s in Big Data?

🧠 The **V’s** describe the **challenges and characteristics of Big Data**

⚡ Core 3 V’s (Most Important)

1 Volume (Size of Data)

📦 **How much data is generated**

- GB → TB → PB → EB
- Massive storage requirement

🌍 Real Example:

- Social media storing billions of posts
- Banking storing millions of transactions

⚠ Problem:

- Cannot store in single machine

🧠 Key Insight:

🧠 **Volume = Scale problem**

---

2 Velocity (Speed of Data)

🧠 **How fast data is generated and processed**

- Real-time or near real-time data

🌍 Real Example:

- Stock market updates
- Payment transactions
- Sensor data (IoT)

⚠ Problem:

- Data arrives continuously and very fast

🧠 Key Insight:

🧠 **Velocity = Speed problem**

---

3 Variety (Different Types of Data)

🧠 **Different formats of data**

Types:

- Structured → Tables
- Semi-structured → JSON, XML
- Unstructured → Images, videos, logs

🌍 Real Example:

- YouTube → video + comments + metadata
- E-commerce → product info + reviews + images

⚠ Problem:

- Hard to process multiple formats

🔑 Key Insight:

🔑 **Variety = Complexity problem**

🔹 Additional V's (Advanced – Good for Interviews)

---

4 Veracity (Data Quality)

🔹 **Is the data reliable and clean?**

🔹 Example:

- Missing values
- Duplicate records

🔹 Insight:

🔹 Garbage data → Wrong insights

---

5 Value (Business Benefit)

🔹 **Is the data useful?**

🔹 Example:

- Customer data → recommendations
- Logs → system improvement

🔹 Insight:

🔹 Data is useless without value

🧠 Easy Memory Trick

🧠 **3 Main V's:**

- Volume = Size
- Velocity = Speed
- Variety = Type

 Extra V's:

- Veracity = Quality
- Value = Benefit

Real-World Mapping

V	Meaning	Example
Volume	Huge data	Billions of records
Velocity	Fast data	Live transactions
Variety	Different types	JSON, video, logs
Veracity	Data quality	Missing/dirty data
Value	Usefulness	Business insights

Interview Insight

If asked:

 "Explain V's of Big Data"

Say:

 "Big Data is characterized by Volume, Velocity, and Variety — referring to the size, speed, and different formats of data. Additionally, Veracity ensures data quality, and Value represents the usefulness of the data for business decisions."

⚠ Common Mistakes

✔Only explaining definitions Always give:

- Meaning
- Example
- Problem

🎯 Final Master Line

🔥 **Big Data = Large (Volume) + Fast (Velocity) + Complex (Variety)**

🤔 Why do we need Spark or Hadoop?

🧠 One-Line Answer

🔥 **We need Spark/Hadoop to process Big Data efficiently using distributed systems**

🧠 The Core Problem

You already learned:

🔥 Big Data = **Huge + Fast + Complex** 🔥 Traditional systems = 💥 Fail

So we need: ✔ Systems that can handle:

- Large data (Volume)
- Fast processing (Velocity)
- Different formats (Variety)

⚙️ What Spark & Hadoop Actually Do

🧠 They allow:

```
Split data → Process in parallel → Combine results
```

✔ Instead of 1 machine → use **many machines**

Hadoop (Storage + Batch Processing)

🧠 What it does:

- Stores huge data (HDFS)
- Processes data in batches (MapReduce)

🧠 Internal Idea:

```
Big file → split into blocks → stored across machines
```

📌 Use Case:

- Log processing (daily)
- Historical data analysis

⚠ Limitation:

- Slow (disk-based processing)

⚡ Spark (Fast Processing Engine)

🧠 What it does:

- Processes data **very fast**
- Uses **memory (RAM)** instead of disk

🧠 Internal Idea:

```
Data loaded into memory → parallel processing → faster results
```

📌 Use Case:

- Real-time analytics
- Machine learning pipelines

🧠 Why Not Just Use Python?

```
data = load_big_data()
process(data)
```

🧠 Problems:

- Runs on **single machine**
- Memory overflow
- Very slow

⚡ Spark Example Thinking

Instead of:

```
1 machine → 1 hour
```

 Spark:

```
100 machines → few seconds/minutes
```

Internal Architecture Thinking

Traditional:

```
Data → Single CPU → Output 
```

Spark/Hadoop:

```
Data → Split → Cluster → Parallel Processing → Output 
```

⚔ Hadoop vs Spark (Important)

Feature	Hadoop	Spark
Speed	Slow	Very Fast
Processing	Disk-based	Memory-based
Use Case	Batch	Real-time + Batch
Complexity	Higher	Easier APIs

Interview Insight

 "Why do we need Spark or Hadoop?"

 "Spark and Hadoop are required to handle Big Data because traditional systems cannot scale. They use distributed computing to split data across multiple machines and process it in parallel, enabling efficient handling of large-scale data."

⚠ Common Mistakes

 Saying “for large data” only

✔ Mention:

- Distributed processing
- Parallel execution
- Scalability

Final Master Line

 Spark/Hadoop = Systems that make Big Data processing possible

---

Monolithic vs Distributed Systems

---

1. Monolithic System

Simple Definition

 Everything runs in a single system (one machine, one codebase)

How it works

```
User → Application → Database → Output
        (All in ONE system)
```

- Single server
- Single CPU / memory
- All components tightly connected

□ Example

```
# Everything in one place
def load_data():
    ...

def process_data():
    ...

def store_data():
    ...
```

□ Real Use Case

- Small applications
- Simple scripts
- Early-stage startups

□ Problems

1. Scalability Issue

□ Can't handle large data

2. Performance Bottleneck

□ One system → overloaded

3. Failure Risk

□ If system crashes → everything stops □

□ 2. Distributed System

□ Simple Definition

□ **System spread across multiple machines working together**

□ How it works

```
Data → Split → Multiple Machines → Parallel Processing → Combine → Output
```

□ Example Thinking

Instead of:

```
1 machine → process all data □
```

We do:

```
100 machines → each handles part □
```

□ Real Use Case

- Big Data processing
- Real-time analytics
- Systems like Spark, Hadoop

□ Advantages

1. Scalability

□ Add more machines easily

2. High Performance

□ Parallel processing

3. Fault Tolerance

□ One machine fails → others continue

⚔ Key Differences (VERY IMPORTANT)

Feature	Monolithic System	Distributed System
Architecture	Single machine	Multiple machines
Scalability	Limited □	High □

Feature	Monolithic System	Distributed System
Performance	Slower for big data	Fast (parallel)
Fault tolerance	Low	High
Complexity	Simple	Complex

Internal Thinking (Engineer View)

Monolithic:

```
All work → One CPU → Sequential processing
```

Distributed:

```
Work split → Multiple CPUs → Parallel processing
```

Interview Insight (Strong Answer)

 “Monolithic vs Distributed?”

 “A monolithic system runs as a single unit on one machine, which limits scalability and performance. In contrast, a distributed system spreads computation across multiple machines, enabling parallel processing, scalability, and fault tolerance, making it suitable for big data workloads.”

⚠ Common Mistakes

❌ Saying only “single vs multiple systems” ✅ Also mention:

- Scalability
- Fault tolerance
- Parallel processing

Final Master Line

 Monolithic = Single brain Distributed = Multiple brains working together

---

🔗 Real Connection (IMPORTANT)

🔹 Monolithic → Traditional systems 🔹 Distributed → Big Data tools (Spark, Hadoop)

🧠 What are “Resources”?

🔹 **Resources = things a system uses to run**

Main resources refers to:

- CPU (processing)
- Memory (RAM)
- Storage (disk)
- Network

🖥 1. Resources in Monolithic System

🧠 Core Idea

🔹 **All resources belong to ONE machine**

⚙ Resource Usage

1 CPU

- Single CPU (or limited cores)
- All tasks share same processor

---

2 Memory (RAM)

- Fixed RAM
- If data is large → 💥 memory overflow

---

3 Storage

- One disk (HDD/SSD)

- Limited capacity

---

4 Network

- Minimal usage
- Mostly internal processing

Resource Flow

```
All tasks → Single CPU + RAM + Disk
```

Problems

- Resource bottleneck
- Cannot scale beyond machine limit
- Overload → system crash

---

2. Resources in Distributed System

Core Idea

Resources are spread across MANY machines (cluster)

⚙ Resource Usage

1 CPU

- Multiple CPUs across machines
- Parallel execution

---

2 Memory (RAM)

- Combined memory of all machines
- Can handle huge datasets

---

3 Storage

- Distributed storage (data split across nodes)

---

4 Network (VERY IMPORTANT)

- Machines communicate over network
- Data transfer happens continuously

Resource Flow

```
Task split → Multiple CPUs + RAM + Disks → Network coordination → Output
```

Advantages

- Scalable (add more machines)
- High performance
- Fault tolerant

⚔ Key Comparison

Resource	Monolithic System	Distributed System
CPU	Single machine	Multiple machines
Memory	Limited	Scalable (cluster)
Storage	One disk	Distributed storage
Network	Low usage	High (communication)

Internal Thinking (Important)

Monolithic:

```
CPU (1) → RAM (1) → Disk (1)
```

Distributed:

```
CPU (many) + RAM (many) + Disk (many) + Network
```

Real Example

Netflix-like system

- Monolithic: Cannot handle millions of users
- Distributed: Uses many servers Scales globally

Interview Insight

“Resources in monolithic vs distributed?”

“In a monolithic system, all resources like CPU, memory, and storage are confined to a single machine, leading to limitations in scalability. In a distributed system, these resources are spread across multiple machines, allowing parallel processing, higher scalability, and better performance, with network playing a key role in coordination.”

⚠ Common Mistake

✔ Ignoring network in distributed systems Network is **critical** (data movement between nodes)

Final Master Line

Monolithic = Limited resources (one machine) Distributed = Scalable resources (many machines)

What is a Scalable System?

Simple Definition

A scalable system is one that can handle increasing load (data/users) without performance issues

---

🧠 Real Meaning (Think Like Engineer)

📌 When:

- Users ↑
- Data ↑
- Requests ↑

✔ System should: ✔ Still work fast ✔ Not crash 🚫 Handle growth smoothly

---

⚙ Types of Scaling

1 Vertical Scaling (Scale Up)

📌 Increase power of same machine

- More RAM
- Better CPU

📌 Limited & expensive

---

2 Horizontal Scaling (Scale Out) 🔥 (IMPORTANT)

📌 Add more machines

```
1 server → 10 servers → 100 servers
```

✔ Used in Big Data systems

🧪 Example

📌 App grows from:

- 100 users → 1 million users

📌 Scalable system:

- Adds servers
- Handles load smoothly 🚀

📌 Non-scalable:

- Crashes 💥

□ Why Monolithic System is NOT Scalable

□ Core Reason

□ **It depends on a single machine**

⚙ Problems in Detail

1 Only Vertical Scaling

- Can only increase:
  - RAM
  - CPU

□ But:

- Has a limit
- Very costly

---

2 No Horizontal Scaling

□ Cannot easily add multiple machines

```
1 system → still 1 system □
```

---

3 Resource Bottleneck

- CPU overloaded
- Memory full
- Disk slow

□ Everything competes for same resources

---

4 Tight Coupling

- All components connected tightly
- Hard to scale individual parts

---

5 Single Point of Failure

□ If system crashes: □ Entire application down

□ Internal Comparison

Monolithic:

```
More load → Same machine → Overload → Crash □
```

Scalable (Distributed):

```
More load → Add machines → Load distributed → Smooth □
```

□ Real Example

E-commerce App

- Festival sale (huge traffic)

□ Monolithic: □ Website crashes

□ Scalable system: □ Adds servers □ Handles traffic

□ Interview Insight (Strong Answer)

□ "What is scalability and why monolithic is not scalable?"

□ "A scalable system is one that can handle increasing workload by adding resources without affecting performance. Monolithic systems are not scalable because they rely on a single machine and support only vertical scaling, which has limitations in terms of cost and capacity, making them unsuitable for large-scale applications."

⚠ Common Mistakes

□ Saying "monolithic cannot scale"

✔ Say **why**:

- Only vertical scaling
- Resource limits
- No distribution

Final Master Line

Scalable = Can grow with demand

Monolithic = Limited growth (single machine)

Why Monolithic System is NOT Scalable?

One-Line Answer

Because it depends on a single machine and cannot grow beyond its limits

---

Deep Understanding

A **monolithic system = everything in ONE place**

- One server
- One CPU
- One memory
- One codebase

When load increases → this single system gets overloaded

⚙ Exact Reasons (Very Important)

1 Depends on Single Machine

```
All requests → One system
```

If users increase: Same machine must handle everything

---

2 Only Vertical Scaling (Big Limitation)

 You can only:

- Increase RAM
- Upgrade CPU

 Problems:

- Very expensive
- Has maximum limit

 You **cannot upgrade forever**

---

3 No Horizontal Scaling

 Ideal scalable system:

```
Add more machines → distribute load
```

 Monolithic:

```
Still 1 machine 
```

---

4 Resource Bottleneck

- CPU becomes busy
- Memory gets full
- Disk becomes slow

 All components compete for same resources

---

5 Tight Coupling (Important for Interviews)

 All parts are connected

- Cannot scale only one part
- Must scale entire system

 Inefficient

---

6 Single Point of Failure

 If system crashes: Entire application goes down

Suppose:

- 100 users → system works fine
- 1,00,000 users →

 Monolithic system: Slow Crashes Cannot handle load

⚖ Comparison Thinking

Monolithic:

```
More load → Same machine → Overload → Failure 
```

Scalable System:

```
More load → Add machines → Load shared → Smooth 
```

 “Why monolithic is not scalable?”

 “Monolithic systems are not scalable because they rely on a single machine and support only vertical scaling, which is limited by hardware constraints. They cannot distribute workload across multiple systems, leading to resource bottlenecks and performance issues as demand increases.”

⚠ Common Mistake

❌ Saying: “because it is single system” Say:

- No horizontal scaling
- Resource limitation
- Tight coupling

Final Master Line

Monolithic fails because it cannot distribute load — everything stays on one machine

---

What is a Distributed System?

Simple Definition

A distributed system is a collection of multiple machines (nodes) that work together as one system

---

Real Understanding

Instead of:

```
1 machine does all work
```

We do:

```
Many machines share the work
```

Each machine handles **a small part of the task**

⚙ How It Works (Internal Flow)

```
Data / Request
      ↓
Split into parts
      ↓
Sent to multiple machines (nodes)
      ↓
Processed in parallel
      ↓
Results combined
```

```
      ↓
Final output
```

🧠 Real Example

📱 Instagram / Banking System

- Millions of users
- Huge data

🧠 Distributed system:

- Different servers handle:
  - User data
  - Messages
  - Payments

✔ Works smoothly even at scale

🧠 Why Distributed Systems are Scalable

🧠 One-Line Answer

🧠 **Because we can add more machines to handle increasing load**

🧠 Key Reasons

---

1 Horizontal Scaling (Most Important)

🧠 You can add more machines:

```
10 servers → 100 servers → 1000 servers
```

✔ Load is distributed ✔ No single system overload

---

2 Parallel Processing

 Multiple machines work **at the same time**

```
Task split → processed simultaneously
```

✔ Faster execution ✔ Handles big data easily

---

3 Resource Expansion

 Resources increase automatically:

- CPU ↑
- Memory ↑
- Storage ↑

✔ No limitation like single machine

---

4 Load Distribution

 Work is shared:

```
Heavy load → divided → each node handles small part
```

✔ Prevents bottlenecks

---

5 Fault Tolerance (Bonus Advantage)

✔ If one machine fails: Others continue

⚖ Comparison Thinking

Monolithic:

```
More load → Same machine → Crash 
```

Distributed:

```
More load → Add machines → Smooth handling 
```

🎯 Interview Insight (Perfect Answer)

🔹 “What is distributed system and why scalable?”

🔹 “A distributed system is a group of interconnected machines that work together to process data as a single system. It is scalable because it supports horizontal scaling, allowing us to add more machines to distribute workload, enabling parallel processing and efficient handling of large-scale data.”

⚠ Common Mistakes

❌ Saying only “multiple systems” ✅ Add:

- Parallel processing
- Horizontal scaling
- Load distribution

🧠 Final Master Line

🧠 Distributed system = Many machines acting as one 🔹 Scalable = Add more machines to handle more load

---